INFORME POST-MORTEM DE INCIDENTE & ANÁLISIS DE CAUSA RAÍZ (RCA)

REPORTE DE DISPONIBILIDAD OPERATIVA Y CUMPLIMIENTO DE SLA

ID DE INCIDENTE: `INC-CRASH-1793394435`
SERVICIO AFECTADO: `API Checkout & Microservicio de Pagos`
SEVERIDAD: `CRÍTICO (P1)`
AUTOR DEL INFORME: `Eugenio Cortés (DevOps Lead)`
ESTADO ACTUAL: `RESUELTO Y MITIGADO`

1. RESUMEN EJECUTIVO

El día **30 de Octubre de 2026 a las 21:05 UTC (18:05 CLT)**, el servicio de Checkout y procesamiento de pagos experimentó una degradación crítica con errores **HTTP 502 Bad Gateway**. Los clientes que intentaban procesar órdenes recibieron fallos de conexión por saturación de hilos en el proxy upstream.

El sistema **GMA-DEV** detectó la falla a los **22 segundos** del primer error 502, alertando al equipo vía Discord y Telegram. El servicio fue completamente restablecido en **9 minutos y 42 segundos** mediante un escalado dinámico de contenedores y liberación del pool de conexiones de base de datos.

2. MÉTRICAS CLAVE DE IMPACTO (SLAs)

Métrica Operativa	Valor Registrado	Meta / Compromiso SLA
Tiempo de Detección (MTTD):	`22 segundos`	`< 1 minuto`
Tiempo de Reconocimiento (MTTA):	`1 minuto 10 segundos`	`< 5 minutos`
Tiempo Medio de Recuperación (MTTR):	`9 minutos 42 segundos`	`< 15 minutos`
Disponibilidad Mensual Afectada:	`99.98%` (Permanece dentro del SLA contractual de 99.95%)	`≥ 99.95%`
Transacciones de Pago Frustradas:	`14 transacciones` (Reintentadas exitosamente tras el reinicio)	`0 críticas`

3. CRONOLOGÍA DE LOS HECHOS (TIMELINE)

- **21:05:00 UTC:** Pico inusual de tráfico concurrente hacia el endpoint `/v1/checkout/transbank`.
- **21:05:15 UTC:** La base de datos agota el pool máximo de conexiones (`max_connections = 100`). Los nuevos hilos de Node.js quedan en espera indefinida.
- **21:05:22 UTC:** Nginx devuelve el primer error `HTTP 502 Bad Gateway` por timeout de upstream (10.000 ms).
- **21:05:25 UTC: GMA-DEV dispara alerta crítica a Discord y Telegram** con estado 502 y latencia 10.042

ms.

- **21:06:35 UTC:** El ingeniero de guardia entra a la consola de administración en GCP / Docker.
- **21:08:00 UTC:** Se ejecuta el comando de reciclaje de contenedores y se incrementa el pool de conexiones en Postgres a 250 mediante PgBouncer.
- **21:14:42 UTC:** Healthchecks responden **HTTP 200 OK con latencia 48 ms**. GMA-DEV emite automáticamente la alerta verde **RESOLVED**.

---

4. ANÁLISIS DE CAUSA RAÍZ (LOS 5 PORQUÉS)

1. **¿Por qué falló el Checkout?**
   Porque Nginx devolvió HTTP 502 por timeout de conexión al backend.
2. **¿Por qué el backend no respondió?**
   Porque los trabajadores de Node.js se quedaron bloqueados esperando una conexión libre a PostgreSQL.
3. **¿Por qué se agotaron las conexiones a la base de datos?**
   Porque una consulta no optimizada ejecutó un *Full Table Scan* en la tabla `orders`, reteniendo 80 conexiones simultáneas por más de 8 segundos.
4. **¿Por qué no había límite de tiempo de consulta (*query timeout*)?**
   Porque la configuración del cliente ORM no tenía definido un `statement_timeout` estricto para peticiones de checkout.
5. **Causa Raíz Definitiva:**
   Falta de un pooler intermedio de conexiones (PgBouncer) y ausencia de índices adecuados en la columna `transaction_id` de la tabla de órdenes.

---

5. PLAN DE ACCIÓN Y MEDIDAS PREVENTIVAS

Acción Correctiva	Responsable	Fecha Límite	Estado
Configurar `statement_timeout = 3000ms` en backend	Equipo Backend	2026-11-02	`En Progreso`
Desplegar PgBouncer para multiplexar conexiones Postgres	Eugenio Cortés (DevOps)	2026-10-31	`Completado`
Crear índice compuesto en `orders(transaction_id, created_at)`	DBA / Backend	2026-11-01	`Completado`
Añadir alerta de saturación de pool > 80% en GMA-DEV	Eugenio Cortés (DevOps)	2026-11-03	`Pendiente`